

GaN Based DC–DC Power Converter with an Improved Gate Driver

Student Name(s):

Sanjay Kumar 23BEC1447 · Aamir Abdullah 23BPS1197 · Amritha S 23BEC1368

Guide:

Dr. Bindu — SENSE

Programme / School:

B.Tech — SENSE, VIT Chennai

Review & Date:

Review-I · 09.09.2026 (Wednesday)

Guide's Signature with Date

Signature: _____

Date: _____

Approval is mandatory before every review. After signing, scan this slide and keep the scanned copy as Slide 1 of the presentation.

Review-I — 09.09.2026 (5 Marks, 5%)

- Focus: Proposed Solution & 50% Work Completion.
- Duration: 10 minutes.
- Presentation contents:
 - Signed & scanned title slide (Slide 1)
 - Problem statement and background
 - Literature survey — minimum 8–10 recent references
 - Existing solutions and their limitations
 - Proposed solution, scope and methodology
 - 50% work completed, timeline, tools & technologies
- Mark split-up: Problem clarity & relevance (2), Literature survey quality (1), Proposed solution feasibility (1), Presentation quality (1).

Problem Statement & Background

- **Problem Statement:**

- When one GaN transistor switches, its fast voltage swing pushes charge into the gate of the other transistor — the one that is supposed to stay OFF. That gate rises to **1.65 V**, and **only 1.4 V is needed to turn it on**. Both devices conduct at once and the supply shorts through them.

- **Background & Significance:**

- GaN half-bridges sit inside EV inverters and battery chargers. GaN is chosen because it switches fast — and that speed is exactly what causes this fault.

- **Existing Solutions:**

- Gate drivers that turn the device on in steps, clamp the off gate down, adjust the gap between the two switching edges, and hold the gate at -2 V [1]–[4]. Reported: 30.5 % less overshoot, 75 % less turn-off loss [5]–[7].

- **Limitations / Research Gap:**

- **Nobody has measured what the re-tuning is worth.** Those gains mix two different things: picking one good setting, and changing the setting while the converter runs. Only the second needs a sensor, a lookup table and a controller.

Aim, and how we approached it

- **Aim:**

- To find out how much of a gate driver's benefit needs the driver to re-tune itself while the converter is running, and how much comes from choosing one good setting and leaving it fixed.

- **Proposed Solution:**

- A GaN buck converter driven by a **segmented gate driver**: 8 pull-up steps, 8 pull-down steps, an adjustable dead time, a Miller clamp and a -2 V off rail — every one of them a setting we can change and measure.

- **How we approached it:**

- **1. Build the converter and check it converts.** 100 V DC in, 48.6 V DC out at 4.88 A — 236.9 W delivered, 97.6 % efficient.
- **2. Recreate the fault.** At the fastest setting the gate that should be OFF reaches 1.65 V, against a 1.4 V turn-on threshold.
- **3. Fix it one change at a time,** measuring each change on its own run, not all at once.
- **4. Ask whether one fixed setting is enough,** by running the driver at two operating points and seeing whether the best setting moves.

- **Scope and tools:**

- **In scope:** the converter, the driver, its settings, and the FPGA controller. **Out of scope:** building hardware. **Tools:** ngspice for every simulation, LTspice to draw the circuit and re-check, Vivado for the FPGA.

What a GaN HEMT is

What a GaN HEMT is

A power transistor built in gallium nitride instead of silicon. Same job, different physics.

Silicon MOSFET			GaN HEMT	
Bandgap	1.1 eV		Bandgap	3.4 eV
Breakdown field	0.3 MV/cm		Breakdown field	3.3 MV/cm — about 10x
Structure	vertical		Structure	lateral, conducts in a 2DEG
Body diode	yes — it conducts backwards		Body diode	none
Gate threshold	typically 3-4 V		Gate threshold	1.4 V in our device
<i>thicker drift region for the same voltage</i>			<i>thinner, lower resistance, far less charge to move</i>	

Because the breakdown field is about ten times higher, the same 200 V rating needs a much thinner conducting layer. Less charge has to be moved to switch the device, so it switches in nanoseconds instead of tens of nanoseconds. That speed is the reason to use GaN — and it is also the reason this project exists, because a faster edge is a bigger disturbance to everything around it.

Device modelled: EPC2010C class — 200 V, 25 mΩ, threshold 1.4 V (models/egan.lib, written from datasheet quantities)

The figure. Silicon against gallium nitride, side by side, on the properties that matter for switching.

What it means. GaN holds off the same voltage in a much thinner layer, so there is far less charge to move and it switches in nanoseconds. That speed is the reason to use it.

Why the GaN HEMT causes the problem we solve

Why the GaN HEMT causes the problem we are solving

Three properties of the device, and what each one does in a half-bridge.

1

The gate turns on at 1.4 V

A silicon MOSFET needs 3–4 V. Ours needs 1.4 V, so there is less than half the room between "off" and "accidentally on". Any disturbance on the gate matters more.

2

There is a capacitor from drain to gate — $C_{GD} = 150$ pF

It is unavoidable: it is the physical overlap inside the device. When the drain voltage moves, charge flows through it into the gate. A 100 V change in a few nanoseconds pushes real current.

3

There is no body diode

A silicon device freewheels through its body diode during the dead time. GaN has to conduct backwards through its own channel, and the drop is $V_{th} + |V_{gate, off}| + I \cdot R_{ds(on)}$ — so holding the gate more negative costs more loss. That coupling is why the -2 V rail is not free.

Put together: the device switches fast, its gate has little margin, and there is a capacitor delivering charge to that gate every time the other device switches. That is the crosstalk fault, and it is a consequence of choosing GaN rather than a mistake in the circuit.

The figure. Three properties of the device and what each one does inside a half-bridge.

What it means. A 1.4 V threshold, an unavoidable 150 pF path from drain to gate, and no body diode. The fault follows from the device, not from a mistake.

The base paper we build on

Takayama, Okuda & Hikihara (2022)	
What it does	Drives the gate from a multibit CODE instead of a resistor — the driver is a DAC, and the code changes DURING the switching edge.
Why it matters to us	A digital code is finite. That is what makes the setting space searchable, and what makes the driver implementable on an FPGA rather than as an analogue network.
What it leaves open	It never asks whether the code has to CHANGE as load, voltage and temperature move. That question is this project.

H. Takayama, T. Okuda and T. Hikihara, “**Digital Active Gate Drive of SiC MOSFETs for Controlling Switching Behavior,**” Int. J. Circuit Theory Appl., vol. 50, no. 1, pp. 183–196, 2022. doi.org/10.1002/cta.3136

The Five Closest — and the base paper we replicate

Ref	Author(s) & Year	Title / Source	Aim — what the paper sets out to do	Gap we found → what WE fill
BASE	H. Takayama, T. Okuda & T. Hikiyama (2022)	Digital Active Gate Drive of SiC MOSFETs for Controlling Switching Behavior — Preparation Toward Universal Digitization of Power Switching Int. J. Circuit Theory Appl. , vol. 50, no. 1, pp. 183–196 doi.org/10.1002/cta.3136	A DAC-inspired driver: a multibit gate signal SEQUENCE sets the gate waveform digitally, so switching behaviour is chosen by a code rather than by a resistor.	THE BASE PAPER. Its multibit gate code is the direct ancestor of our 720-point control word, and it is digital — implementable on an FPGA — rather than a fixed analogue network. We replicate its premise on GaN and then ask the question it does not: of the gain a code buys, how much needs per-operating-point adaptation at all? WE FILL: the exhaustive 720-word search that makes the fixed-vs-adaptive split measurable, and the GaN third-quadrant cost their SiC device never pays.
10	Zhang, Yu, Leng, Cui, Deng & Ng (2020)	A Segmented Gate Driver for E-mode GaN HEMTs with Simple Driving Strength Pattern Control Proc. IEEE ISPSD, 2020 , pp. 102–105 ieeexplore.ieee.org/document/9170108	Segmented output stage on E-mode GaN: 7 slices, pattern timing 0.5–5 ns, strength set by one external bias resistor	Architecturally the closest published driver to ours — segmented slices, pattern-selected strength. It is an ASIC with a fixed pattern set; our contribution is to search the whole pattern space exhaustively and price what the search actually buys. WE FILL: the price of the pattern space: what an exhaustive search buys over their fixed pattern set.
11	Wang, Tao, Xiao, Luo, He, Zhou, Zhang & Wang (2024)	High-Frequency Three-Level Gate Driver for GaN HEMT Bridge Crosstalk Suppression IEEE Trans. Power Electron. , 39(1), 1343–1352 ieeexplore.ieee.org/document/10286072	Three-level drive; capacitor–diode negative rail with a digitally-clamped zero level, to 5 MHz	Recent, GaN, and on exactly our failure mode. It suppresses crosstalk with added passives; we get the same protection from the Miller clamp already in the cell and measure its price at 0.04 %. WE FILL: whether the setting has to change with the operating point at all. It does not: 3.9 % of baseline.
12	Wang, Sun, Yan, Ma & Xu (2024)	An Integrated Suppression Method of Both Gate-Source Voltage Oscillation and Crosstalk for GaN HEMT Gate Driver IEEE Trans. Power Electron. , 39(11), 14643–14655 ieeexplore.ieee.org/document/10591431	One driver addressing gate-source ringing and crosstalk together	Confirms the two effects are coupled, which is why our cost function prices loss and overshoot jointly rather than optimising either alone. WE FILL: both effects in ONE cost function, showing the objectives genuinely conflict (Pareto, slide 16).
13	Cai, Ye, Lv & Chen (2026)	A High-Efficient GaN Driver With Hybrid Adaptive Dead-Time Control and Peak Delay Control for Synchronous Buck Converter IEEE Trans. Power Electron. , 41(1), 279–290 ieeexplore.ieee.org/document/11146698	Hybrid adaptive dead-time plus peak delay control on a GaN synchronous buck	The most recent adaptive dead-time driver we found, and it adapts the one field our leave-one-out test shows carries the whole benefit — and only at light load. WE FILL: which field is worth scheduling. Only dead time, and only at light load; drive strength is worth 0.00 %.

[9] is the BASE PAPER: we reproduce its premise — a gate waveform chosen by a multibit code — on GaN, then extend it. All five are verified against the publisher record.

We implemented the base paper, then ours

Citing a base paper is not a comparison, so we built theirs too. **models/basedrv.lib** is Takayama, Okuda & Hikiyama's driver: a multi-bit code that changes DURING the switching edge, with no gate clamp and no -2 V rail, because those two are ours. It runs inside the same sim/dpt.cir, so only the driver differs.

Base paper, as we built it	+0.534 V	Their changing-in-time code on its own already clears the 1.4 V threshold.
Ours, constant code, no clamp	−0.249 V	FALSE TURN-ON. A fast fixed code is worse than their timed one — their idea is real, and we reproduce it.
Ours, active Miller clamp on	+0.570 V	Only just past the base paper. The clamp on its own is not the story.
Ours, clamp + -2 V off-bias	+2.576 V	4.8× the base paper's margin. This is the version we ship.

What the comparison shows. The base paper shapes the gate over TIME; we keep the code steady and add two things they do not have. Both clear the threshold — theirs works — but ours clears it by 4.8× more, and ours is the one whose settings can then be searched in full.

Command: python3 scripts/basepaper_compare.py — four ngspice runs, prints this table.

The gap this project fills

Every paper on these drivers reports ONE number: how much better it is than a plain driver.
That number hides two separate effects, and they cost very different hardware.

Effect 1 — a better FIXED setting

Pick one good setting once, at design time. Costs nothing while running: no sensor, no ADC, no lookup table, no controller.

25.1 %
of baseline

Effect 2 — ADAPTING per operating point

Change the setting as load, voltage and temperature move. This is the part that needs the sensing hardware.

3.9 %
of baseline

THE GAP: nobody separates them. No paper says how much of the benefit actually needs the re-tuning, because separating the two means running every setting at every operating point — and nobody has done that.

WHY IT MATTERS: only Effect 2 pays for the sensor, ADC and lookup table. If it is small, that hardware is mostly buying something a design-time choice already gives you.

WHAT WE DO: 720 settings × 4 operating points, all of them, in ngspice — then report the two numbers separately instead of adding them together.

The driver's settings, and where 720 comes from

What the driver's "settings" are, and where 720 comes from

Six things the gate driver can be told to do. Every combination is one setting; every setting is one simulation.

FIELD	VALUES WE SWEEP	HOW MANY	WHAT IT CHANGES
Pull-up slices	1, 2, 3, 4, 6, 8	6	how hard the device is switched ON
Pull-down slices, low side	2, 8	2	how hard it is switched OFF
Pull-down slices, high side	1, 4, 8	3	how firmly the OFF device's gate is held down
Dead time	5, 10, 15, 25, 35 ns	5	the gap before the other device turns on
Miller clamp	off, on	2	a switch that shorts the gate down while the device is off
Off-bias rail	0 V, -2 V	2	what voltage the gate is parked at when off

$6 \times 2 \times 3 \times 5 \times 2 \times 2 = 720 \text{ settings}$

Each one is simulated at each of four operating points, so the full search is 2,880 runs. That is what lets us say a setting is the best one, rather than the best one we happened to try. The list is in scripts/sweep.py; the hardware that produces these fields is rtl/seg_gate_ctrl.v, and it emits exactly these six.

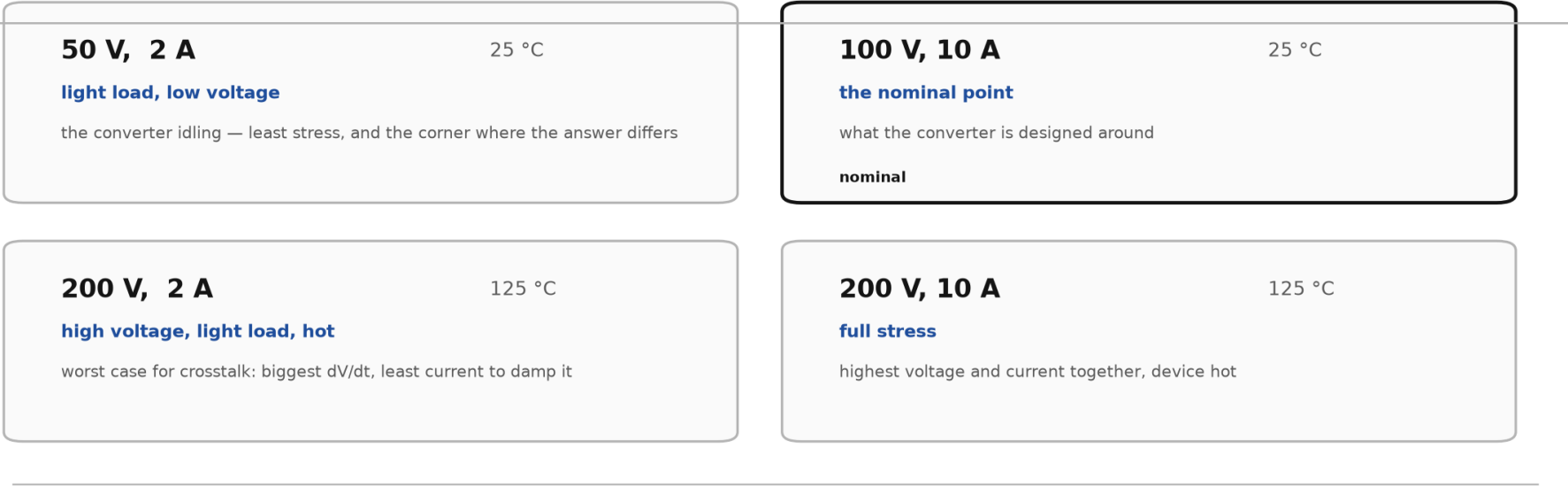
The figure. The six fields the gate driver exposes, and the values each one is swept over.

What it means. $6 \times 2 \times 3 \times 5 \times 2 \times 2 = 720$ settings. Each is simulated at four operating points, so 2,880 runs in all.

The input — what an operating point is

What we mean by the input, or "operating point"

A converter does not run at one condition. These are the four we test every setting at.



Three things change between them: the bus voltage the converter is fed, the current the load draws, and the junction temperature of the devices. Temperature matters because it raises on-resistance and lowers the turn-on threshold — the model does both.

Every one of the 720 settings is simulated at all four. The question the project answers is whether the best setting is the same at all four, or whether it moves — because only if it moves is there anything for a controller to adapt to.

The figure. The four conditions every setting is tested at: bus voltage, load current and junction temperature.
What it means. If the best setting were the same at all four there would be nothing to adapt to. Whether it moves is the question the project answers.

The output — what is actually measured

What comes out

Two different things are called "the output" in this project. Both are measured, neither is estimated.

1. THE CONVERTER'S OUTPUT		2. WHAT ONE SIMULATION REPORTS	
what the machine delivers to its load		measured on every one of the 2,880 runs	
Output voltage	48.56 V DC	Peak on the OFF gate	the crosstalk voltage
Output current	4.875 A	Margin	1.4 V minus that peak
Power delivered	236.85 W	Switching energy	turn-on + turn-off + dead time
Power drawn	242.63 W	Overshoot	how far past the bus the node rings
Efficiency	97.62 %	Settling time	how long the ringing lasts
Ripple on the output	729 mV	Oscillation energy	30-500 MHz content, for EMI

The left column is the converter working: 100 V DC in, 48.56 V DC out, 97.62 % of the power getting through. That is the deliverable.

The right column is what each individual simulation measures, and it is how settings are compared. The definitions are fixed in one file (scripts/gansim.py) and were frozen before the sweeps ran, so no measurement was redefined after seeing a result.

Both reproduce on this machine: `python3 scripts/bucksim.py` and `python3 scripts/cases.py`

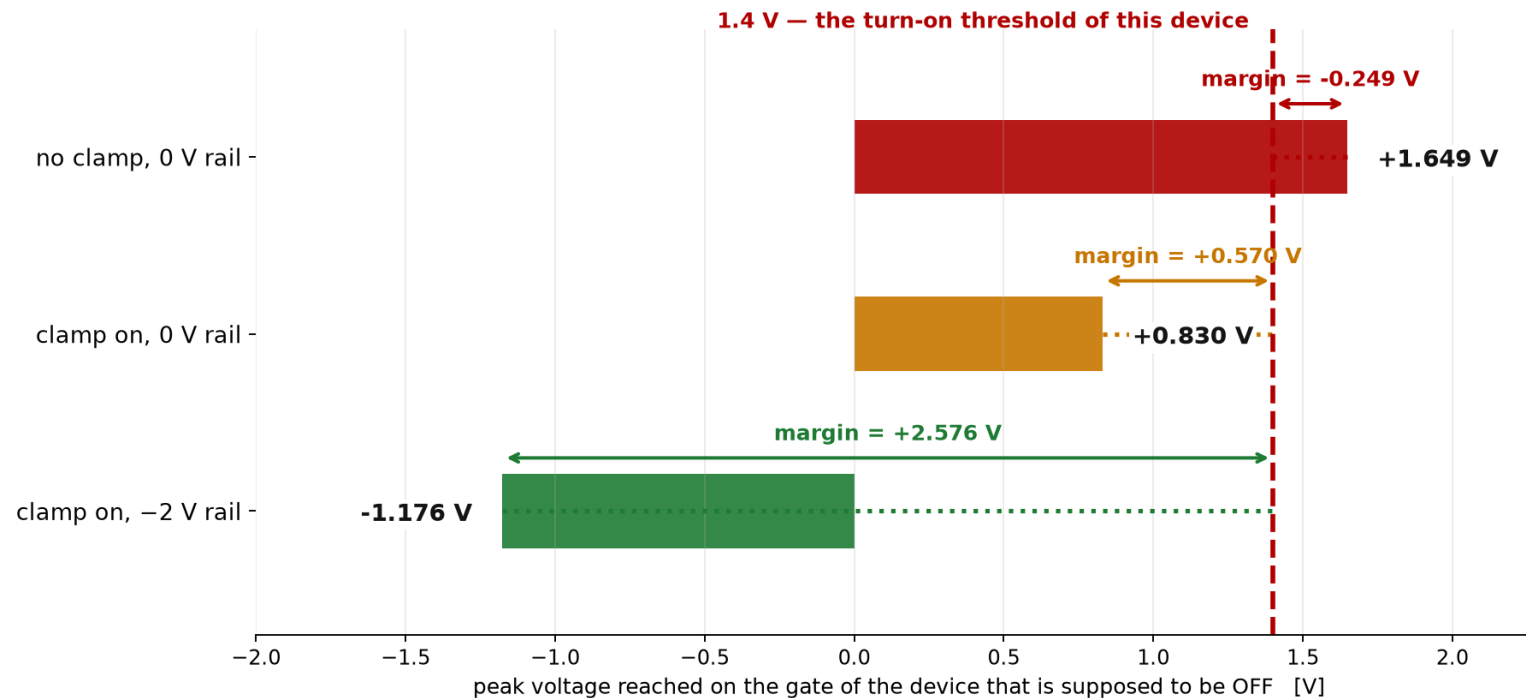
The figure. Two things: what the converter delivers, and what each individual simulation reports.

What it means. 48.56 V at 4.875 A, 97.62 % efficient. Per run: the peak on the OFF gate, the margin, the switching energy and four more.

What “margin” means

What "margin" means

Margin is one number: how far the OFF device's gate stays below the voltage that would switch it on. Positive is safe, negative is a fault.



Negative margin is not a small error — it means both devices conduct at once and the supply is shorted through them. 2.58 V of margin is the shipped design.

The figure. How far the OFF device's gate stays below the voltage that would switch it on, for three configurations.

What it means. Positive is safe, negative is a fault. Without the fix it is -0.249 V; with the clamp and the -2 V rail it is +2.576 V.

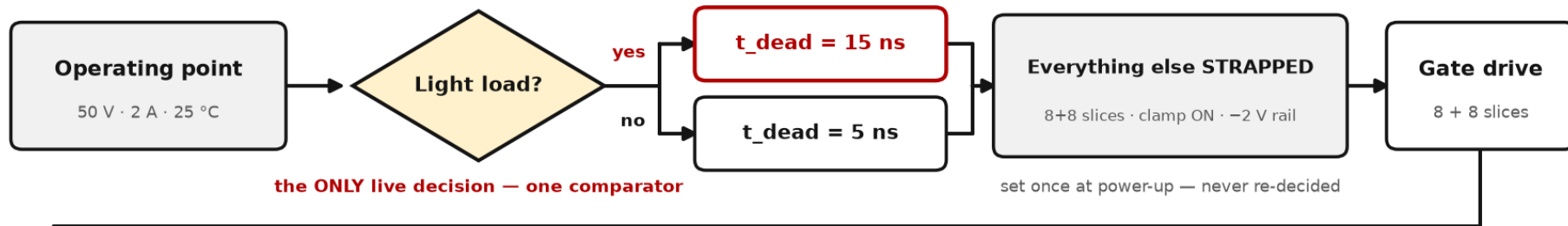
How it works — one use case, start to finish

USE CASE

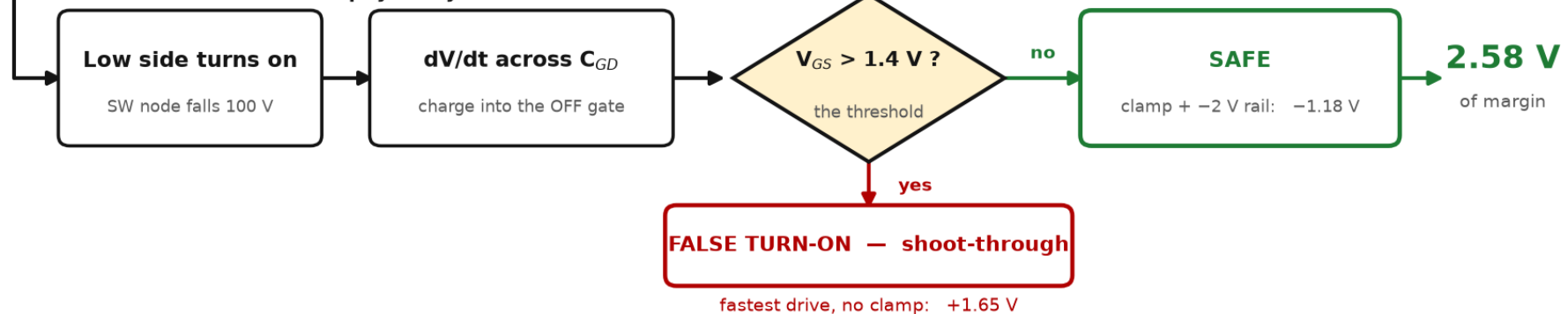
Battery-storage converter — load falling 10 A → 2 A as the pack nears full charge.

Light load is the only one of the four corners where the best dead time differs. This is where the adaptive question is decided.

EVERY SWITCHING EDGE — what the controller decides



IN THE CIRCUIT — what physically follows

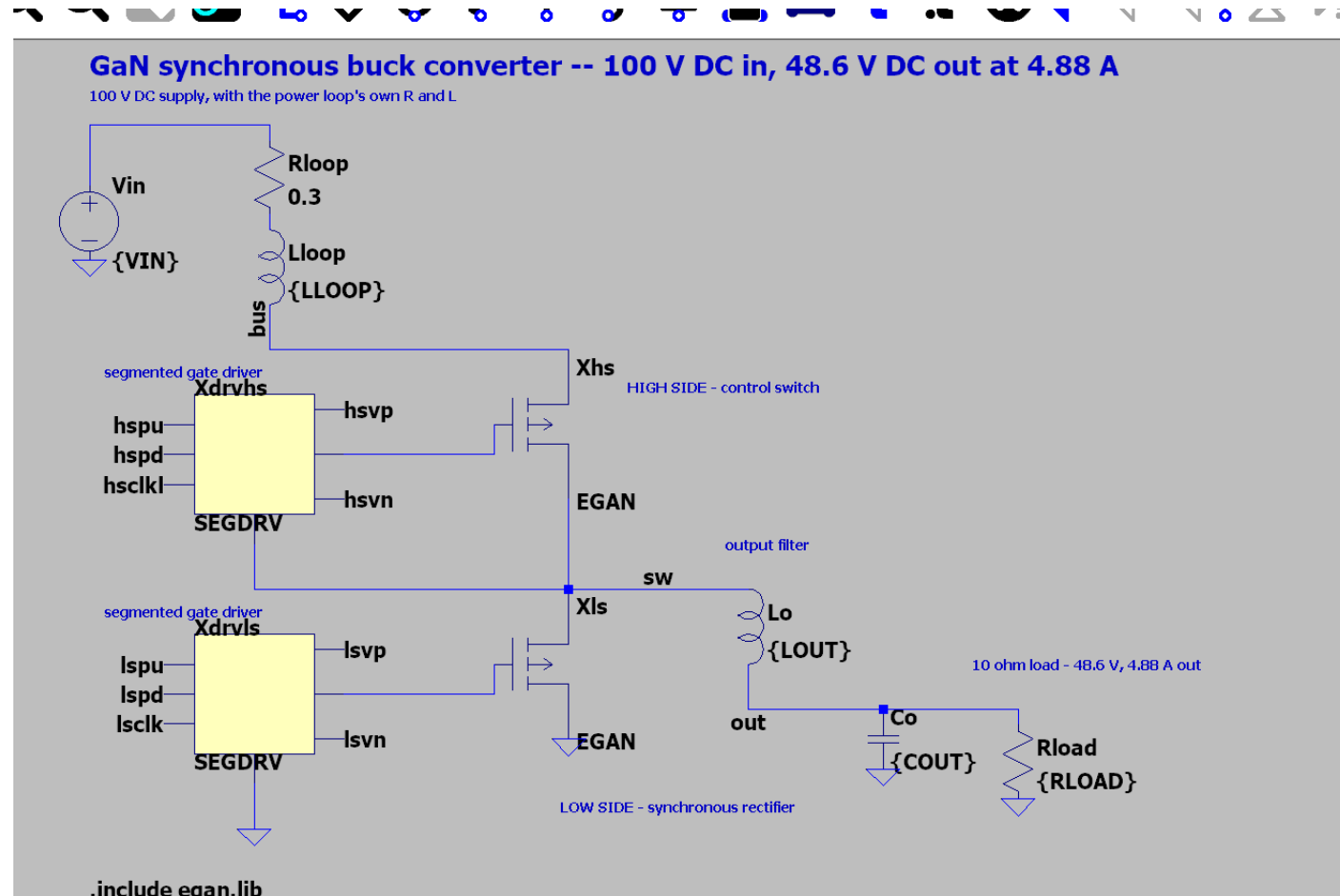


Everything adaptive reduces to the one shaded decision. Freezing it costs 5.45 % across four corners — and 0.00 % if the light-load corner is left out.

The figure. One switching edge followed through. Top row: what the controller decides. Bottom row: what the circuit then does.

What it means. The one shaded diamond is the only live decision. Everything else is set once at power-up.

The circuit, drawn and simulated



The figure. Itspice/BUCK_converter.asc as LTspice draws it: the 100 V supply and its loop parasitics, the two GaN HEMTs, a segmented driver on each gate, the output filter and the load.

What it means. This is not a picture of a circuit — it is the circuit. Press Run and it gives 48.84 V and 4.88 A, against the netlist's 48.56 V and 4.875 A.

How the work was run

How the work was actually run

Six steps, in this order. Each one had to hold before the next was worth doing.

1	Build the device and the driver A GaN HEMT written from datasheet numbers, and a segmented driver with the six fields.
2	Recreate the fault Drive it as fast as possible with nothing to stop crosstalk, and confirm the OFF gate crosses 1.4 V.
3	Fix it one change at a time Clamp, then off-bias rail, then drive strength — each measured on its own run, not all together.
4	Search every setting at every operating point $720 \text{ settings} \times 4 \text{ operating points} = 2,880 \text{ runs}$, so the best setting is the best, not the best tried.
5	Separate the two effects How much comes from choosing one good fixed setting, and how much from re-tuning it while running.
6	Check it somewhere else The same circuit drawn in LTspice, and the controller built in Verilog and synthesised in Vivado.

Steps 1–3 are on one switching edge, where the fault lives and can be measured precisely. Steps 4–6 are the study built on top of it.

The figure. The six steps, in the order they were done.

What it means. Each step had to hold before the next was worth doing. The fault is recreated before anything is claimed to fix it.

Which tool did what

Which tool did what

Two tools for the circuit. Every number in this project is produced by ngspice; LTspice draws the same circuit and re-measures the result everything else rests on.

ngspice

every circuit simulation in the study

- **The converter**
sim/buck.cir — 100 V DC in, 48.6 V DC out, 97.6 % efficient
- **The switching test bench**
sim/dpt.cir — one switching edge, measured precisely
- **Every setting, every corner**
720 driver settings $\times$ 4 operating points = 2,880 runs
- **The named cases**
13 runs: the fix built one change at a time
- **Robustness**
21,600 runs across five device parameters
- **Board inductance and EMI**
7,200 runs across eight loop inductances

about 35,000 transient runs in total

LTspice

the drawing, and one check

- **The circuit, drawn**
ltspice/*_design_*.asc — open it and press Run
- **An independent check**
the same three cases, in a second simulator

agrees with ngspice to within 2 mV

The FPGA controller is separate work: written in Verilog, simulated in Icarus Verilog, synthesised in Xilinx Vivado (20 LUTs, 200 MHz met).

The figure. Every circuit simulation in the study is ngspice. LTspice draws the same circuit and re-measures the crosstalk result.

What it means. One tool does the work. The second exists so that no headline number rests on a single program.

What Python does

What Python does here

Python runs no circuit maths of its own. It drives the simulators, collects what they return, and does the arithmetic on top.

It drives ngspice

- writes a netlist for each setting
- runs ngspice and waits
- reads the raw output back
- extracts the eight measurements
- repeats it 2,880 times

scripts/gansim.py, sweep.py

It does the arithmetic

- finds the best setting per point
- finds the best single fixed setting
- the gap between them is the answer
- sweeps the weighting to check it holds
- writes the CSVs the figures read

scripts/ceiling.py, novelty.py

It draws every figure

- reads the CSVs back
- and the LTspice .raw files
- no number is retyped by hand
- every figure regenerates from data

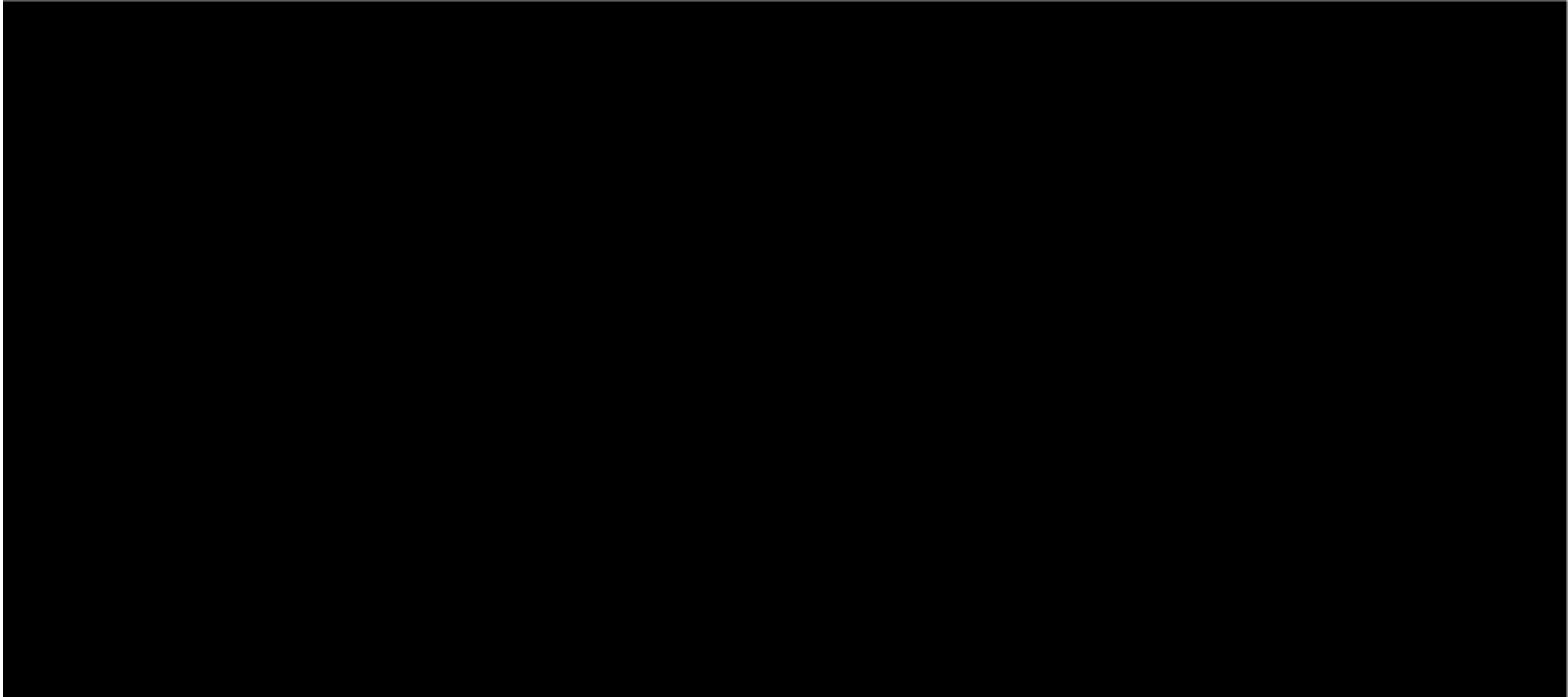
*scripts/*_figure.py*

So if Python had a bug, the circuit results would not change — they come out of ngspice. What would change is which setting we called best. That is why the decomposition is checked a second way: MATLAB and GNU Octave re-derive the same split from the same CSVs.

The figure. The three jobs Python has: driving ngspice, doing the arithmetic on what comes back, and drawing the figures.

What it means. No circuit maths is done in Python. A bug there could change which setting we call best, but not the voltages ngspice reports.

Demo — ngspice and LTspice, running

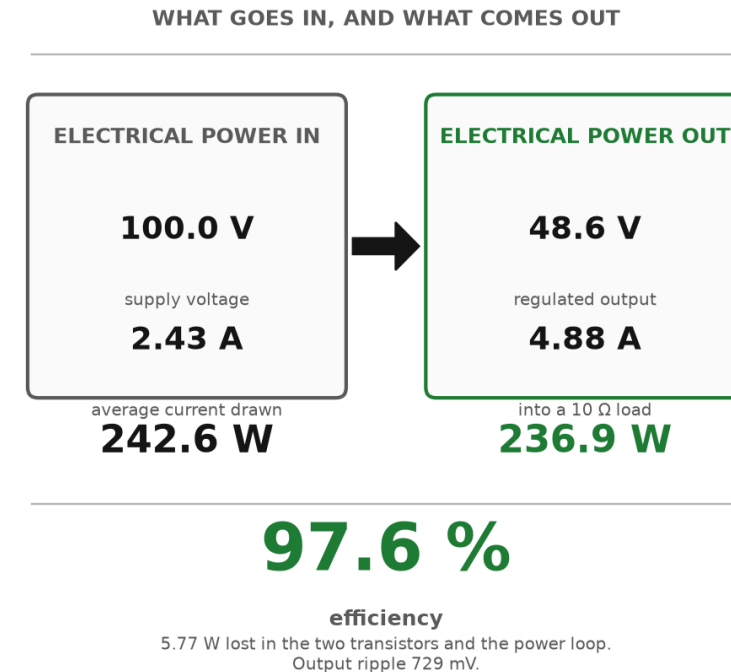
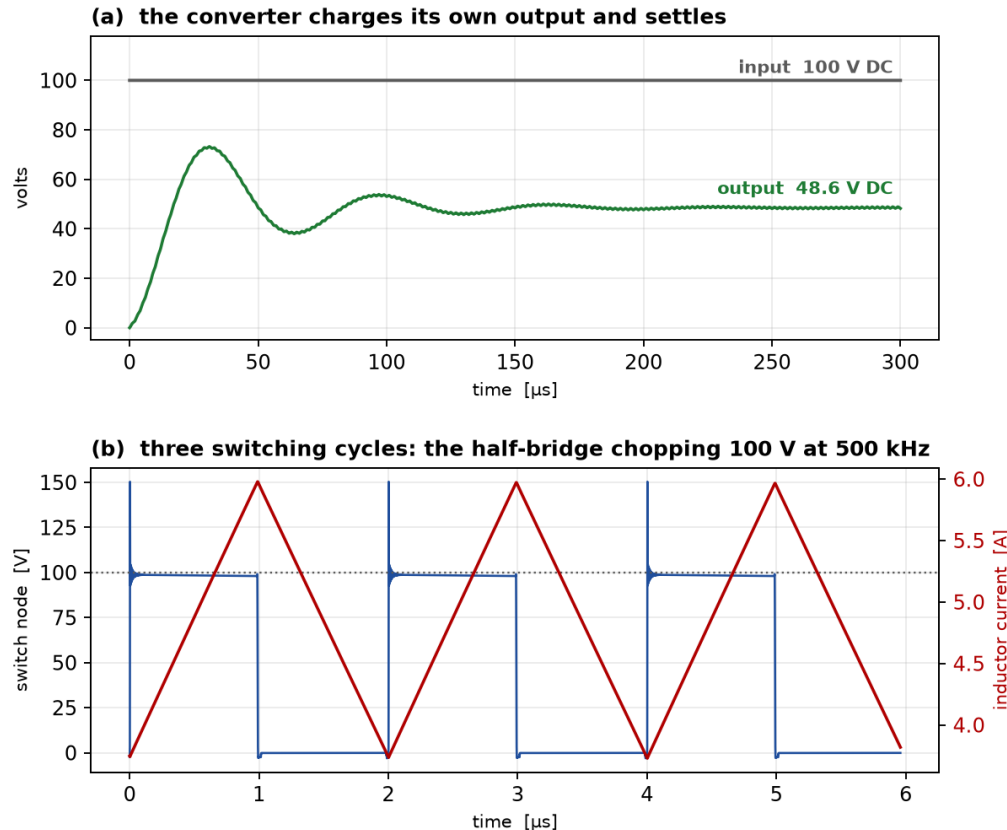


The figure. Two minutes recorded on the project laptop. The converter in ngspice, the crosstalk fault and its fix, the named cases, the Verilog controller, then LTspice opening the schematic and running it.

What it means. Every number in this deck appears in that recording, produced by a tool while the clock in the corner runs. Click the frame to play.

What we are building — the converter

GaN synchronous buck converter — sim/buck.cir, ngspice • 100 V DC → 48.6 V DC at 4.88 A



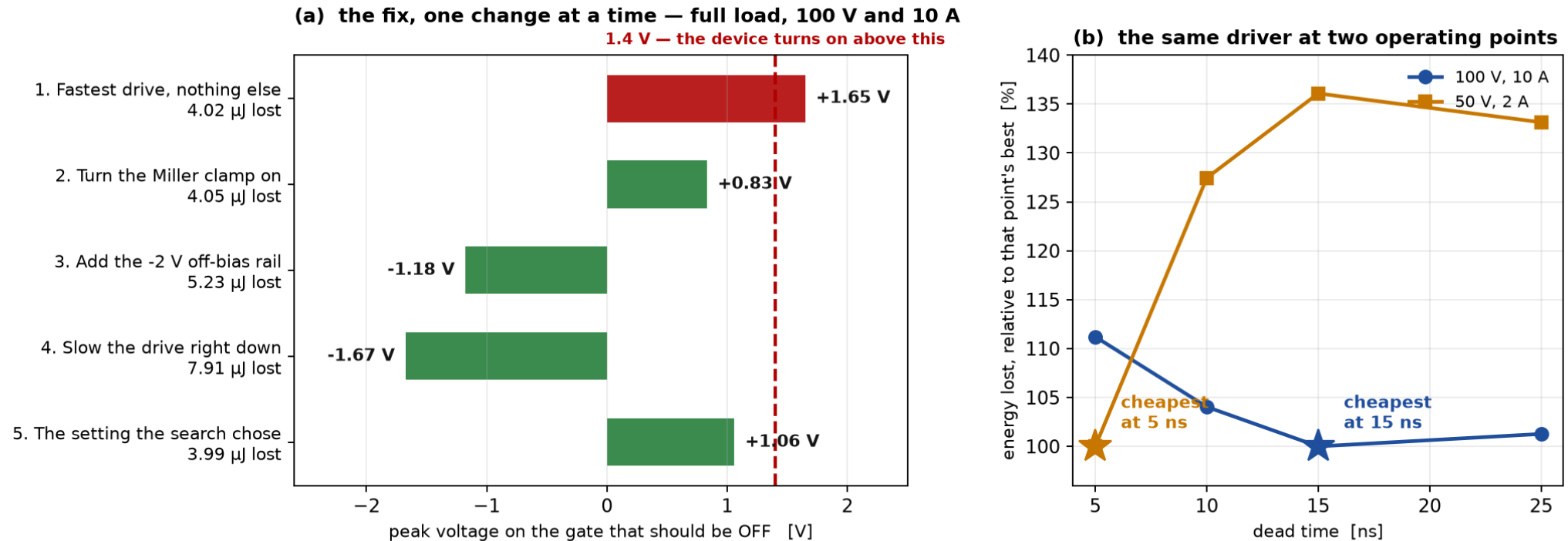
The figure. (a) the output charging from zero and settling; (b) three switching cycles of the half-bridge; (c) power in against power out.

What it means. The converter works. 100 V DC becomes 48.6 V DC at 4.88 A, and 97.6 % of the power gets through.

The cases we ran, one at a time

Named cases, each a separate ngspice run of sim/dpt.cir • python3 scripts/cases.py

Left: turning the clamp on removes the fault; the -2 V rail buys margin; slowing the drive is safest but wastes $2\times$ the energy.
Right: the cheapest dead time is 15 ns at full load and 5 ns at light load. Those are different numbers, and that difference is the only thing worth adapting.

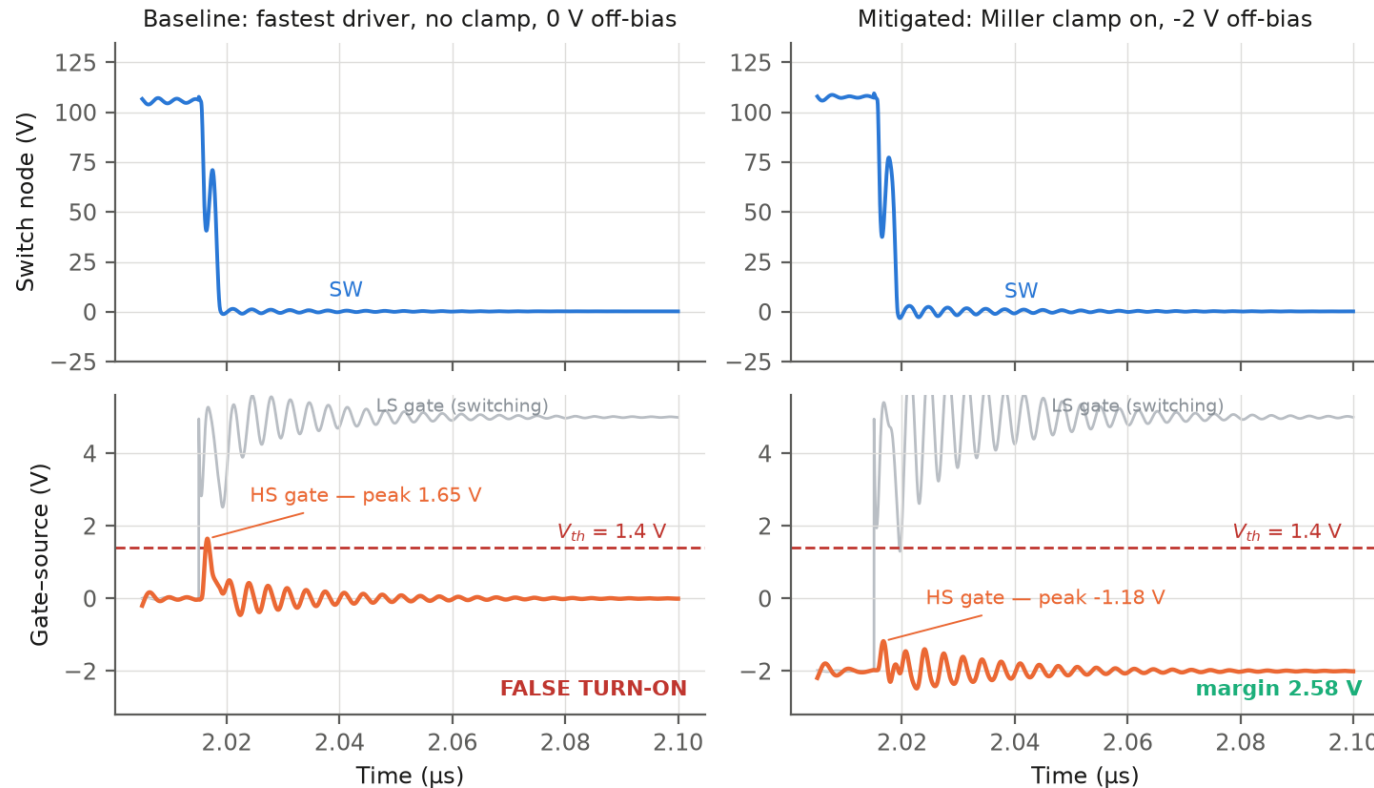


The figure. (a) five settings at full load, each its own ngspice run; (b) the same driver swept at two operating points.

What it means. The clamp removes the fault. The cheapest dead time is 15 ns at full load and 5 ns at light load — two different numbers, and that gap is the only thing worth adapting.

Result 1 — crosstalk is real; the clamp fixes it

Crosstalk at low-side turn-on, 100 V / 10 A



The figure. Voltage on the gate that is supposed to be OFF, with the fix and without.

What it means. 1.65 V against a 1.4 V threshold turns the device on. The clamp and the -2 V rail leave 2.58 V of margin.

The problem is real, and reproduced

1.65 V

peak on the gate that should be OFF

The threshold is 1.4 V — so it turns on by mistake

2.58 V

margin with the clamp and the -2 V rail

The fault is removed completely

And safety is nearly free

The lowest-energy setting is already safe at three of the four operating points. At the fourth, safety costs 0.04 % more energy — 3.335 μ J against 3.334 μ J.

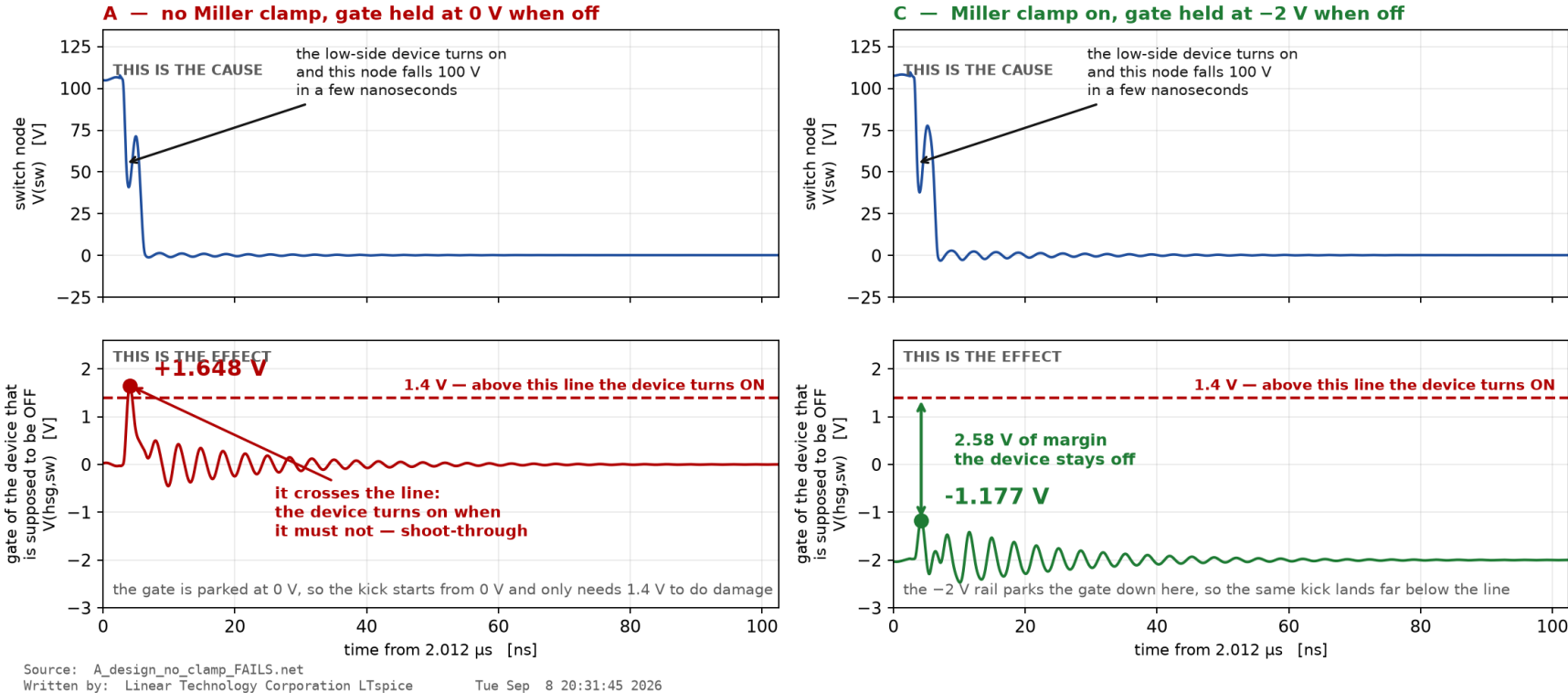
The clamp is worth 9.7–12.2 % of the blended cost, with no sensor and no controller.

The same result in LTspice, on the drawn circuit

LTspice output — redrawn from its own .raw file so it can be labelled

This is not a screenshot. The traces are LTspice's data, read out of the file it wrote, and replotted so the meaning can be marked on it. The screenshots of LTspice itself are alongside.

Same circuit both sides; the only difference is the Miller clamp and the off-bias rail. LTspice measures +1.647556 V and -1.176857 V — ngspice measures +1.6486 V and -1.1757 V on the same netlist.



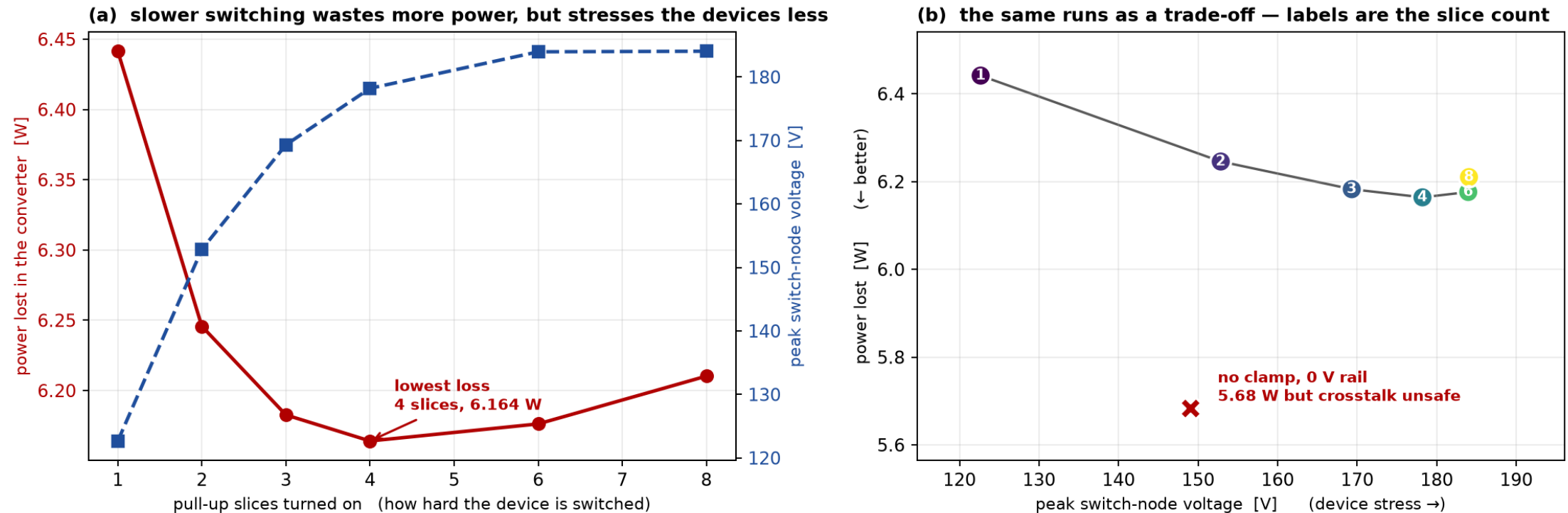
The figure. LTspice's own output, from the schematic in ltspice/. Top: the switch node falling 100 V. Bottom: what that does to the gate of the device that is supposed to be OFF.

What it means. A second simulator, on a drawn circuit, gets +1.6476 V and -1.1769 V where ngspice gets +1.6486 and -1.1757. About a millivolt apart, so the result is the circuit's, not the simulator's.

Result 2 — the driver setting moves the converter

The gate-driver setting, measured on the running converter — 8 ngspice runs of sim/buck.cir

Loss moves 4.5 % and peak device voltage moves 50 % across the same knob. Crosstalk safety costs 0.53 W, or 0.22 points of efficiency.



The figure. Power lost and peak device voltage against drive strength, measured on the running converter.

What it means. They move in opposite directions, so no setting is best at both. Lowest loss is 4 slices, not the fastest.

Result 3 — re-tuning is worth only 5.2 %

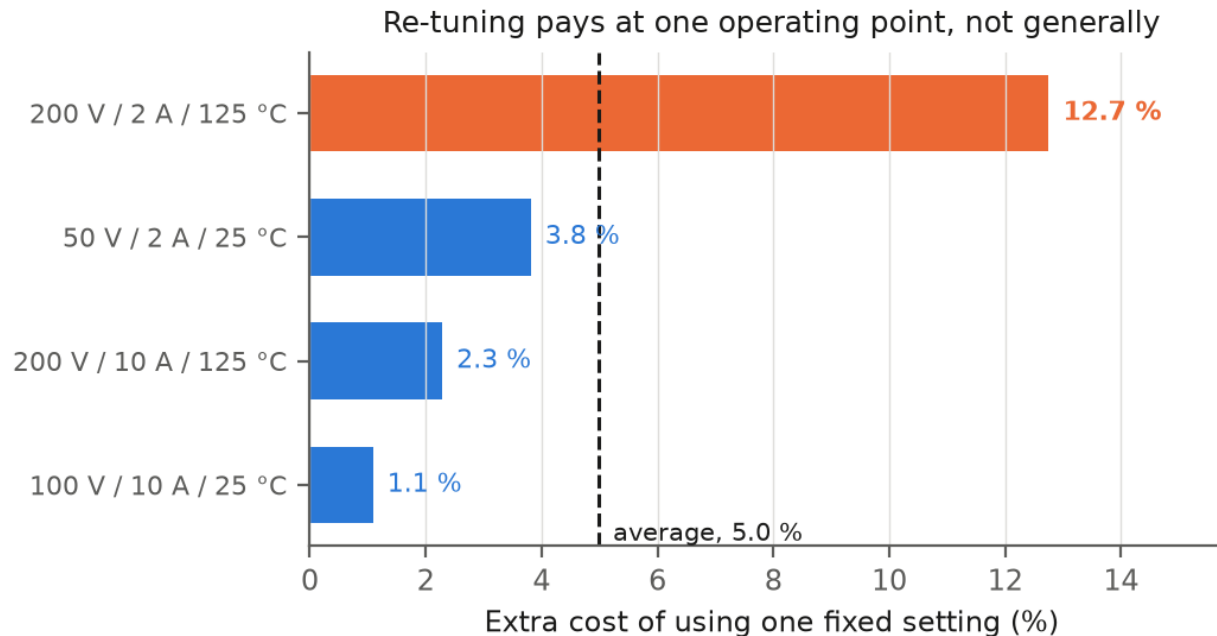


Fig. 2 What it costs to use one fixed setting instead of the best setting at each operating point.

The figure. What one fixed setting costs at each of the four operating points.

What it means. Three points lose 1–4 %. One loses 12.7 %. So re-tuning is worth 5.2 % at the very most.

What re-tuning is actually worth

5.2 %

the most that re-tuning can ever gain

against the best single fixed setting (= 3.9 % of baseline). One fixed setting is nearly as good.

The benefit is not spread out. Three operating points lose only 1–4 % from a fixed setting; one loses 12.7 %.

5.2 % is the generous figure — on a finer 36-point grid it is 2.0 %. It all comes from the dead time, and that from one operating point. Freezing the dead time costs 5.45 % across four points. Drop the light-load 50 V / 2 A point and it costs 0.00 %: three points want 5 ns, only that one wants 15 ns. So the adaptive hardware shrinks to one light-load detector choosing between two dead times. Freezing the drive strength costs 0.00 % — and that is what these papers actually adjust.

Steady against the device, not the board. Across 21,600 runs no device parameter moves the answer outside 4.3–7.7 %. Halving the board inductance takes it to 13.5 %, and that is layout, not the transistor [3].

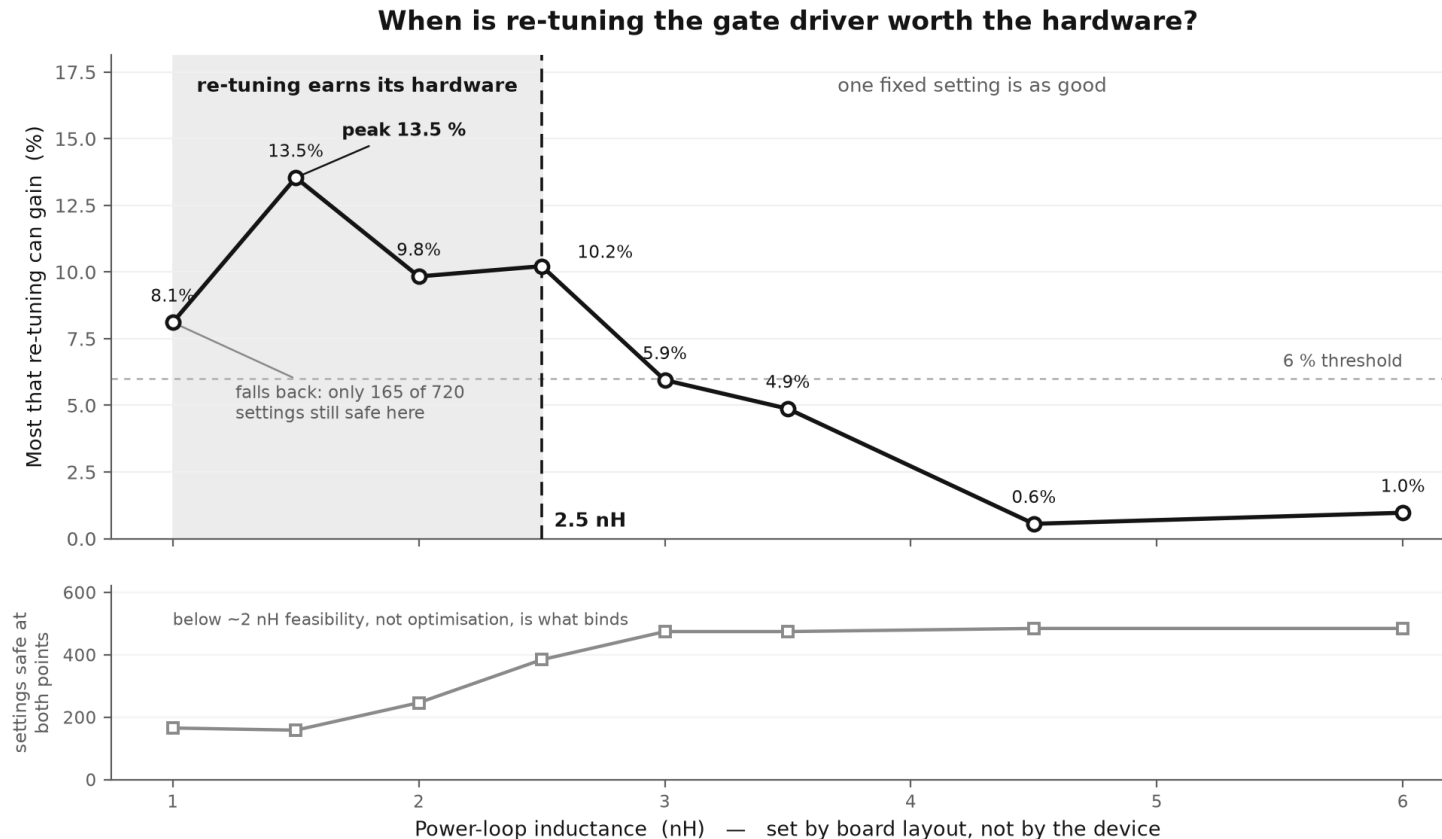
Result 4 — the two halves nobody separates

The base paper [9] (doi.org/10.1002/cta.3136) shows that a gate waveform can be chosen by a digital code. It, and every paper after it, then reports one number — how much better than a plain driver. That number adds together two separate effects, and only the second needs a sensor, an ADC and a lookup table. Separating them means running every setting at every operating point, which is why nobody has.

(A) Pick a better FIXED setting	25.1 %	no sensing · no LUT
(B) CHANGE it per operating point	3.9 %	needs all of it
(B') ...but ONE comparator gets 72 % of (B)	2.8 %	a threshold, not a LUT

What it means. The full sensor + ADC + lookup table is left justifying 3.7 % of the gain, over one fixed setting plus a single comparator. The split does not depend on how the two goals are weighted: across 106 weightings, (A) stays 23.4–29.0 % and (B) 1.3–6.4 %, and (A) wins every time.

Result 5 — re-tuning only pays below ~ 2.5 nH



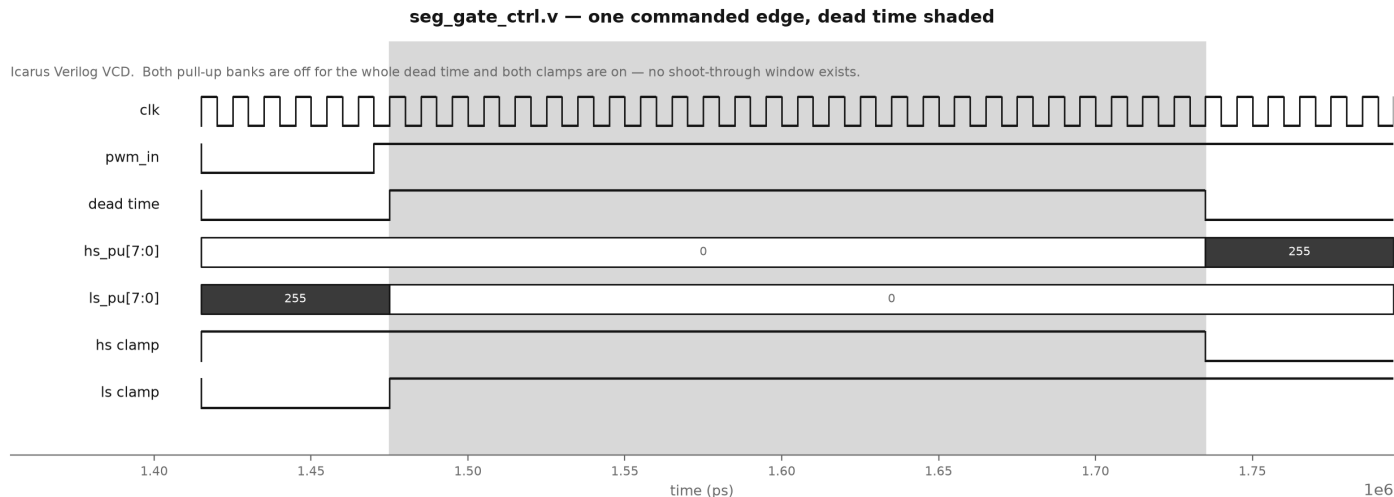
All 720 settings at two operating points for every value, 7,200 runs. Excluding clamp-chatter points changes nothing. `scripts/loop_sweep.py`, `loop_analyse.py`.

The figure. What re-tuning is worth, against the inductance of the power loop. Eight values, 7,200 runs.

What it means. It only pays below about 2.5 nH. Loop inductance is board layout, so this is a layout decision, not a device one.

FPGA Controller — verified RTL

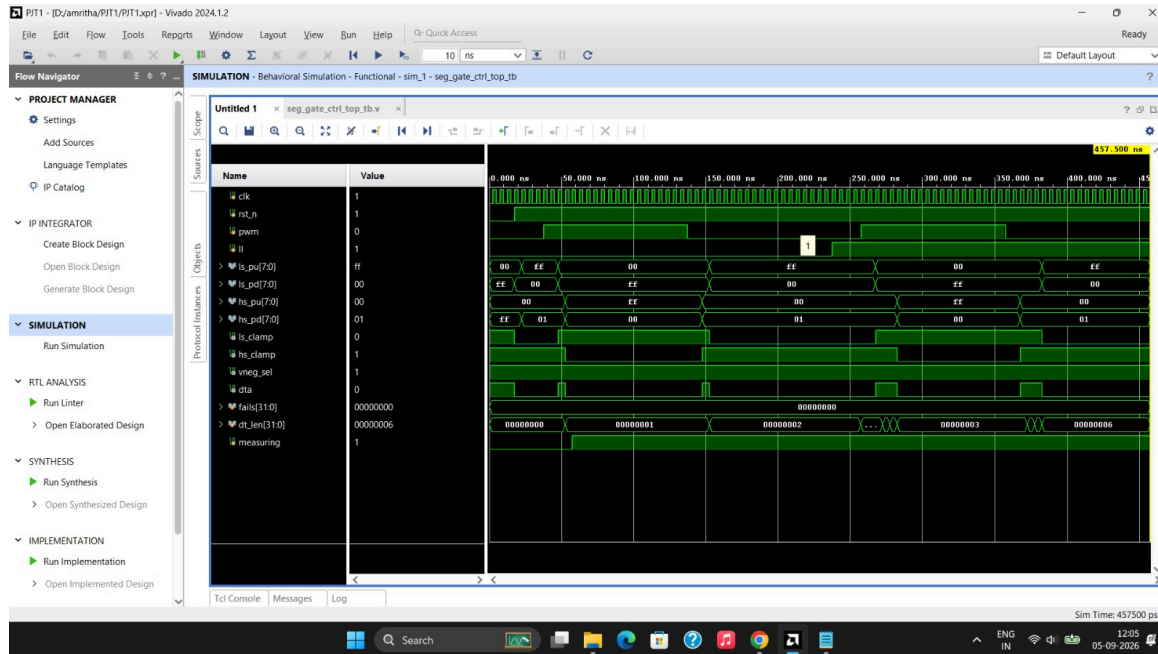
- Three Verilog modules that produce exactly the 720 settings the SPICE model uses.
- Dead time is a **live register**; drive strength is **fixed at power-up** — worth 5.45 % against 0.00 %.
- Eight checks pass in Icarus Verilog, safe reset included. A deliberately broken version is caught 221 times (sh rtl/mutate.sh).
- **Built in Vivado 2024.1.2** on an xc7a35t FPGA: **20 LUTs, 20 flip-flops** — 0.10 % of the chip. 200 MHz timing is **met, with 1.996 ns to spare**.
- **Timing is met where the logic runs.** The paths that miss are chip-output paths, where the output pad alone takes 3.49 ns of a 4 ns budget — pad delay, not logic.



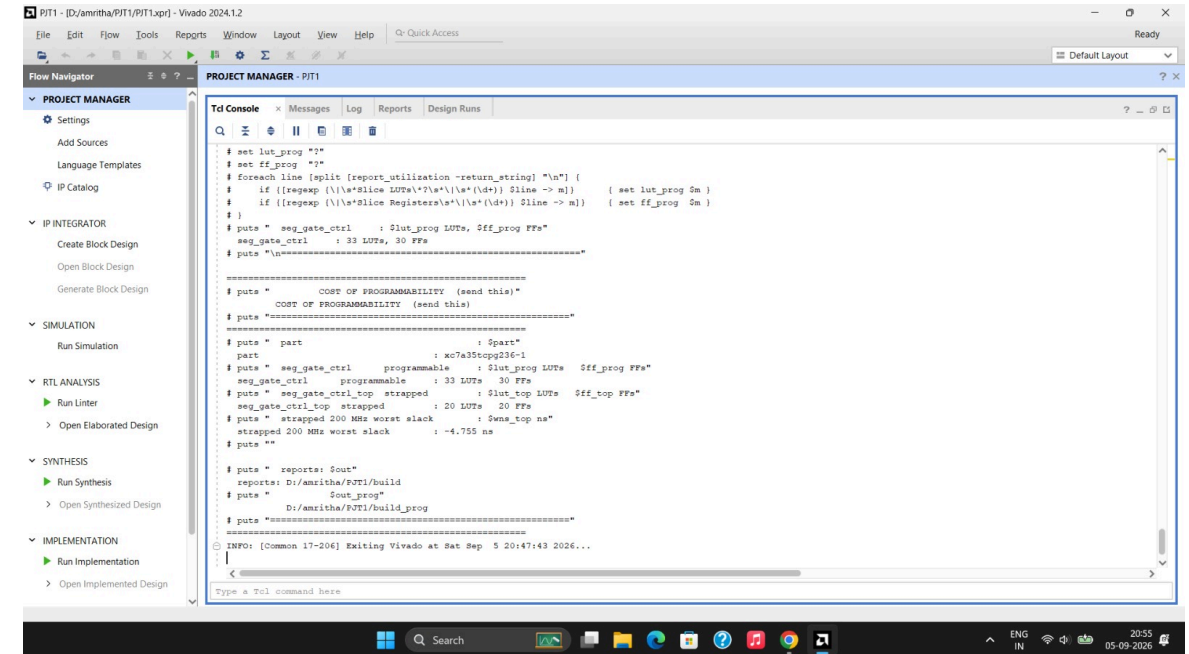
The figure. The controller's own output, captured by Icarus Verilog.

What it means. The two pull-up banks never overlap. The shaded gap is the dead time, so no shoot-through window exists.

Vivado — simulation and synthesis, on screen



Behavioural simulation. fails[31:0] = 0 — every asserted property holds in Vivado's own simulator, a third tool after Icarus and the mutation test.



Synthesis console. 33 LUTs programmable against 20 strapped — the cost of programmability, printed by the tool.

Work Completed — 50 %

- **The converter is built and it runs.**
 - 100 V DC in, 48.6 V DC out at 4.88 A — 236.9 W into the load from 242.6 W drawn, 97.6 % efficient.
- **The fault is reproduced, and fixed.**
 - The off gate reaches 1.65 V against a 1.4 V threshold. With the clamp and the -2 V rail: 2.58 V of margin.
- **Specific cases run and measured, one at a time.**
 - Five settings at full load, then the same driver swept at two operating points — every number below came off its own ngspice run.
- **The setting measured on the converter itself.**
 - Across the same knob, power lost moves 4.5 % and peak device voltage moves 50 %, in opposite directions.
- **The FPGA controller is written and verified.**
 - Eight checks pass in Icarus Verilog; 20 LUTs and 20 flip-flops in Vivado, 200 MHz met with 1.996 ns to spare.

Where we are, and what is next

- **Where we are**

- **Review-I is done.** The converter is built and converting, the crosstalk fault is reproduced and fixed, the named cases have been run, and the FPGA controller is written and verified.

- **What we aim to do next**

- **Review-II — close the loop.** Add a feedback controller so the output holds its value when the load changes, then re-run the driver-setting study with the loop closed.
- **Review-III — settle the light-load question.** The cheapest dead time is 15 ns at full load and 5 ns at light load. Run the converter at both and measure what a two-setting controller actually saves against one fixed setting — that is the number the whole project turns on.
- **Same tools throughout.** ngspice for the circuit, Vivado for the FPGA, start to finish.
- **The risk we already know.** Closing the loop changes where the converter actually operates, so the setting study has to be redone afterwards, not before. That ordering is the schedule risk.

- **Where the work lives**

- github.com/Amritha902/gan-driver — the circuits, the scripts, the Verilog and the Vivado reports.

Conclusion & next steps

What the data supports

- The converter works: 100 V DC in, 48.6 V DC out at 4.88 A — 236.9 W, 97.6 % efficient.
- Picking the setting well: 25.1 %. Changing it per operating point: 3.9 %.
- One comparator gets 72 % of that 3.9 %.
- **What to build:** one fixed setting plus a light-load comparator — not a sensor, an ADC and a lookup table.
The FPGA half is real: 20 LUTs and 20 flip-flops, 200 MHz met with 1.996 ns to spare.
- **What is next**
 - Review-II — close the loop on the converter, then re-run the setting study with it closed.
 - Review-III — measure what a two-setting controller saves at the light-load point, where the best dead time differs.
 - Every number in this deck is regenerated by a named script, in ngspice or in Vivado.

References (1–15)

- [1] Z. Zhang, F. Wang, L. M. Tolbert and B. J. Blalock, "Active Gate Driver for Crosstalk Suppression of SiC Devices in a Phase-Leg Configuration," IEEE Trans. Power Electron., vol. 29, no. 4, pp. 1986–1997, Apr. 2014.
- [2] R. Xie, H. Wang, G. Tang, X. Yang and K. J. Chen, "An Analytical Model for False Turn-On Evaluation of High-Voltage Enhancement-Mode GaN Transistor in Bridge-Leg Configuration," IEEE Trans. Power Electron., vol. 32, no. 8, pp. 6416–6433, Aug. 2017.
- [3] D. Reusch and J. Strydom, "Understanding the Effect of PCB Layout on Circuit Performance in a High-Frequency Gallium-Nitride-Based Point of Load Converter," IEEE Trans. Power Electron., vol. 29, no. 4, pp. 2008–2015, Apr. 2014.
- [4] B. Li, G. Zhang, C. Li, G. Wang, S. Liu and D. Xu, "Crosstalk Suppression Method for GaN-Based Bridge Configuration Using Negative Voltage Self-Recovery Gate Drive," IEEE Trans. Power Electron., vol. 37, no. 4, pp. 4406–4418, 2022.
- [5] X. Chen, H. Peng, S. Song, Q. Tong and Y. Kang, "A Novel Control Strategy for Optimal Tradeoff between Overshoot and Switching Loss Based on Double Closed-Loop Self-Regulating Active Gate Driver," IEEE Trans. Power Electron., vol. 39, no. 10, pp. 13033–13043, 2024.
- [6] W. Song, T. Hu, J. Chen, T. Tang, H. Yue and G. Liu, "A Self-Regulating Active Gate Driver of Voltage Overshoot Suppression for SiC MOSFETs Under Variable Load Current Conditions," IEEE Trans. Power Electron., vol. 40, no. 8, pp. 10623–10634, 2025.
- [7] C.W. Kuo, T.W. Wang, L.C. Chen, C.C. Tu, Y.K. Hsiao and P.H. Chen, "Universal Active Gate Driver IC With Closed-Loop Timing Control and Gate-Sensing Technique for Silicon Carbide Power Devices," IEEE Trans. Power Electron., vol. 40, no. 4, pp. 5120–5129, 2025.
- [8] G.H. Thuc and C.J. Chen, "An Integrated Driver With Dual-Edge Adaptive Dead-Time Control for GaN-Based Synchronous Buck Converter," IEEE Trans. Ind. Appl., vol. 60, no. 6, pp. 9157–9170, 2024.
- [9] **BASE PAPER** — H. Takayama, T. Okuda and T. Hikihara, "Digital Active Gate Drive of SiC MOSFETs for Controlling Switching Behavior — Preparation Toward Universal Digitization of Power Switching," Int. J. Circuit Theory Appl., vol. 50, no. 1, pp. 183–196, 2022.
- [10] W.J. Zhang, J. Yu, Y. Leng, W.T. Cui, G.Q. Deng and W.T. Ng, "A Segmented Gate Driver for E-mode GaN HEMTs with Simple Driving Strength Pattern Control," in Proc. IEEE 32nd Int. Symp. Power Semicond. Devices ICs (ISPSD), 2020, pp. 102–105.
- [11] X. Wang, M. Tao, J. Xiao, D. Luo, M. He, Q. Zhou, X. Zhang and M. Wang, "High-Frequency Three-Level Gate Driver for GaN HEMT Bridge Crosstalk Suppression," IEEE Trans. Power Electron., vol. 39, no. 1, pp. 1343–1352, 2024.
- [12] L. Wang, X. Sun, Y. Yan, M. Ma and D. Xu, "An Integrated Suppression Method of Both Gate-Source Voltage Oscillation and Crosstalk for GaN HEMT Gate Driver," IEEE Trans. Power Electron., vol. 39, no. 11, pp. 14643–14655, 2024.
- [13] Y. Cai, D. Ye, W. Lv and Z. Chen, "A High-Efficient GaN Driver With Hybrid Adaptive Dead-Time Control and Peak Delay Control for Synchronous Buck Converter," IEEE Trans. Power Electron., vol. 41, no. 1, pp. 279–290, 2026.
- [14] Y. Zheng, S. Zhao, P. Agarwal, F. Liu, Y. Zhao and A. Mantooth, "A Multi-Level Turn-Off Gate Driver for Crosstalk Noise Suppression of GaN HEMTs," IEEE Open J. Power Electron., pp. 1–15, 2026.
- [15] M.K. Banda, S. Madichetty, D.M. Natham and S. Koduru, "An Event-Triggered Dual-Polarity Gate Clamp for GaN HEMT Gate Oscillation Suppression," IEEE Trans. Power Electron., vol. 41, no. 9, pp. 14402–14405, 2026.

References (16–30)

- [16] Z. Yuan, H. Li, M. Zhang, Z. Yang, S. Zhao, H. Wang, X. Wang and L. Ding, “Influence of Negative Gate Bias on Crosstalk Spike in SiC MOSFETs Half-Bridge Circuit,” *IEEE J. Emerg. Sel. Topics Power Electron.*, vol. 14, no. 3, pp. 3217–3229, 2026.
- [17] L. Zhang, K. Wang, S. Guo and B. Zhu, “A Gate Driver for Crosstalk Suppression of eGaN HEMT Power Devices,” *J. Low Power Electron. Appl.*, vol. 15, no. 3, pp. 38, 2025.
- [18] J. Wu, “Parallel arrangement of MOSFETs, effective suppression of crosstalk: A new gate driver topology,” *iEnergy*, vol. 2, no. 3, pp. 163–163, 2023.
- [19] Y. Mikhaylov, G. Buticchi and M. Galea, “A gate driver for parallel connected MOSFETs with crosstalk suppression,” *iEnergy*, vol. 2, no. 3, pp. 240–250, 2023.
- [20] W. Liu and H. Min, “A Load Adaptive Active Gate Driver with Fast-Switching Three-Level Current Source for Overshoot Suppression in GaN-Based Half-Bridge Converters,” *IEEE Trans. Power Electron.*, pp. 1–8, 2026.
- [21] R. Chen, F. Wang, H. Bai and L.M. Tolbert, “A Simple Gate Driver Circuit for Turn-on Overvoltage Suppression of 10 kV SiC MOSFETs With Temporarily Lowering Gate Voltage During dv/dt ,” *IEEE Trans. Ind. Electron.*, pp. 1–6, 2026.
- [22] D. Yu, M. Yang, L. Yin, K. Hu, W. Zhang and Q. Fu, “A Dual-Loop Active Gate Driver With Independent Voltage Slew Rate and Overshoot Control for Switching Loss Optimization of SiC MOSFETs,” *IEEE Trans. Ind. Electron.*, pp. 1–12, 2026.
- [23] P. Xiang, R. Hao, J. Cai and X. You, “An Active Gate Driver of SiC MOSFET Module Based on PCB Rogowski Coil for Optimizing Tradeoff Between Overshoot and Switching Loss,” *IEEE Trans. Power Electron.*, vol. 38, no. 1, pp. 245–260, 2023.
- [24] S. Fukunaga, H. Takayama and T. Hikihara, “Slew rate control of switching transient for SiC MOSFET in boost converter using digital active gate driver,” *IET Power Electron.*, vol. 16, no. 3, pp. 472–482, 2022.
- [25] C. Xu, P. Fu, X. Liao, Z. Zhu and L. Liu, “An Integrated Driver With Real-Time Predictive Dead-Time Optimization Technique for GaN-Based Synchronous Buck Converter,” *IEEE J. Emerg. Sel. Topics Power Electron.*, vol. 13, no. 3, pp. 3173–3183, 2025.
- [26] C. Chen, P. Chiu, Y. Chen, P. Wang and Y. Chang, “An Integrated Driver With Adaptive Dead-Time Control for GaN-Based Synchronous Buck Converter,” *IEEE Trans. Circuits Syst. II*, vol. 69, no. 2, pp. 539–543, 2022.
- [27] J. Tan, Z. Zhou and G. Zou, “A Programmable Gate Driver Module-Based Multistage Voltage Regulation SiC MOSFET Switching Strategy,” *Electronics*, vol. 13, no. 22, pp. 4379, 2024.
- [28] C. Chen, P. Wang, S. Li, Y. Chen and Y. Chang, “An Integrated Driver With Bang-Bang Dead-Time Control and Charge Sharing Bootstrap Circuit for GaN Synchronous Buck Converter,” *IEEE Trans. Power Electron.*, vol. 37, no. 8, pp. 9503–9514, 2022.
- [29] G.H. Thuc, M. Tsai, C. Chen, J. Fu, W. Wu and W. Lin, “A Current Source Gate Driver with Dual-Edge Adaptive Slew Rate for GaN Devices,” in *Proc. 2026 IEEE Applied Power Electronics Conference and Exposition (APEC)*, 2026.
- [30] G. Wu, Y. Huang, Y. Chen and C. Chen, “Closed-loop Slew Rate Control of Active Current Source Gate Driver with Digital Implementation for SiC MOSFET,” in *Proc. 2025 IEEE Energy Conversion Conference Congress and Exposition (ECCE)*, 2025.

All 30 verified against the publisher record — authors, volume, issue and pages resolved through Crossref by DOI and title, not transcribed by hand.

Thank you

GaN Based DC–DC Power Converter with an Improved Gate Driver

Amritha S 23BEC1368 · Sanjay Kumar 23BEC1447 · Aamir Abdullah 23BPS1197

Guide: Dr. Bindu — SENSE, VIT Chennai

Everything in this deck runs on request: github.com/Amritha902/gan-driver